

AeroStream Capstone – Real-Time Flight Telemetry Streaming

Introduction

The second phase of the capstone focuses on ingesting and processing high-velocity data using streaming systems. The objective was to calculate rolling aggregations (such as average delays per airline and busiest routes) as data arrived in real time.

The original implementation utilized legacy Spark DStreams (RDD-based streaming). This architecture was completely rewritten to utilize modern **Apache Kafka** and **PySpark Structured Streaming**, providing a much cleaner, SQL-like approach to unbounded data streams.

System Overview

The streaming infrastructure is fully containerized using Docker (`docker-compose.yml`), making it incredibly easy to deploy and scale.

- **The Producer** (`producer.py`): A custom Python Kafka Producer reads the cleaned flight telemetry and streams it directly into a Kafka topic (`flight_telemetry`) in JSON format.
- **The Broker**: A Confluent Kafka broker handles the message queuing, eliminating the need to manually provision EC2 instances with bash scripts.
- **The Consumer** (`consumer.py`): A PySpark Structured Streaming application subscribes to the Kafka topic. It parses the JSON payloads dynamically and treats the unbounded stream as an infinitely growing DataFrame.

Optimizations & Modernization

1. **Structured Streaming vs. DStreams**: The legacy code required complex, manual state management using `updateStateByKey` and custom reduce functions to maintain rolling

averages. By upgrading to PySpark Structured Streaming, this was reduced to highly readable aggregations: `df.groupBy("UniqueCarrier").agg(avg("ArrDelay"))`.

2. **Elimination of Cassandra:** The original project wrote outputs to Cassandra. Modern architectures often write streaming aggregates directly to fast, in-memory dashboards or data lakes. For this implementation, the streams output directly to the console in complete mode for real-time validation.
3. **Topic Partitioning:** The Kafka topic was created with careful attention to partitioning. In a production cloud environment, scaling the partitions allows Spark to parallelize the read operations automatically.

Results

The streaming analytics successfully mirrored the batch processing results, but updated dynamically as new "events" were fired into the Kafka broker.

- The stream correctly identified that HA (Hawaiian Airlines) maintained the lowest rolling delay.
- The stream captured the busiest routes dynamically without the need to query a static database.

Conclusion

Transitioning from RDD-based DStreams to Structured Streaming drastically reduced the codebase footprint while increasing readability and maintainability. The performance overhead of serializing/deserializing Python dictionaries was mitigated by enforcing a strict Spark StructType schema right at the ingestion layer.

This real-time streaming pipeline, combined with the XGBoost model developed in the batch phase, creates a complete, end-to-end MLOps architecture capable of reading live flight telemetry and providing split-second delay predictions.